

FULL STACK DEVELOPMENT

FASE 2 | AULA 04 -

ARQUITETURA MVC COM EXPRESS.JS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	5
SAIBA MAIS	6
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANDA

O QUE VEM POR AÍ?

O padrão arquitetural Model-View-Controller (MVC) é uma abordagem eficaz para organizar o código em desenvolvimento de software. Ele divide uma aplicação em três partes principais:

Modelo (Model)

- Gerencia os dados e a lógica de negócios;
- Armazena, recupera e processa informações.

Visualização (View):

- Cuida da apresentação da interface do usuário;
- Exibe dados e interage com o Modelo.

Controlador (Controller):

- Age como intermediário entre o Modelo e a Visualização;
- Processa entradas do usuário, atualiza o Modelo e reflete as mudanças na View.

Principais benefícios do MVC:

- Separação de responsabilidades: facilita a organização do código, distinguindo claramente as funções do Modelo, View e Controller.
- Reutilização de código: permite utilizar o Modelo em várias partes da aplicação, evitando redundância.
- Flexibilidade e escalabilidade: facilita expansão da aplicação e ajustes sem impactar todo o sistema.

Considerações Importantes

- Curva de aprendizado: pode exigir algum tempo para compreender e implementar corretamente.
- Possível monolitismo nos controladores: os controladores podem se tornar grandes e complexos se não forem gerenciados adequadamente.

- Comunicação em tempo real: pode necessitar de soluções extras para lidar eficientemente com atualizações em tempo real.

EXEMPLO

HANDS ON

Nesta seção prática, embarcaremos em uma jornada interativa para desvendar os segredos de uma API em ação, utilizando a arquitetura MVC. Vamos explorar cada camada em detalhes, desde o Model, onde reside a essência dos dados, até o View, responsável por dar vida à informação. Ao longo do caminho, você colocará a teoria em prática e construirá sua própria API, passo a passo, consolidando seus conhecimentos e se tornando expert na arte da comunicação entre sistemas.



SAIBA MAIS

DRY (Don't Repeat Yourself):

A ideia de que "o código do Modelo pode ser reutilizado em diferentes partes da aplicação, proporcionando maior modularidade e reduzindo a duplicação de código" está alinhada com o princípio de desenvolvimento de software conhecido como "Don't Repeat Yourself" (DRY). Vamos explorar mais sobre essa abordagem e como ela se relaciona com o Modelo em uma arquitetura MVC.

Esse princípio é um conceito fundamental no desenvolvimento de software, que enfatiza a importância de evitar a duplicação de código. A ideia é que cada peça de conhecimento ou lógica na aplicação deve ter uma única representação autoritativa. Evitar a duplicação não apenas reduz o tamanho do código, mas também simplifica a manutenção, minimiza erros e facilita futuras atualizações.

Aplicação no Modelo

Reutilização do código do Modelo

O Modelo, na arquitetura MVC, encapsula a lógica de negócios e a manipulação de dados. Sendo assim, pensando em benefícios, essa lógica pode ser reutilizada em diferentes partes da aplicação. Por exemplo, se você tiver operações de manipulação de dados específicas (como validações ou transformações) ou regras de negócios que são aplicáveis em várias partes da aplicação, elas podem ser encapsuladas no Modelo e reutilizadas sempre que necessário.

Centralização da Lógica

Ao centralizar a lógica de negócios no Modelo, você evita a repetição dessa lógica em diferentes partes do código. Isso não só economiza esforço de desenvolvimento, mas também garante consistência, já que as mesmas regras são aplicadas em todos os lugares onde o Modelo é utilizado. Logo, é um grande benefício.

Facilidade de Manutenção

Outro benefício é que se houver necessidade de alterações na lógica de negócios ou nas operações de dados, essas mudanças podem ser feitas em um único lugar no Modelo. Isso facilita a manutenção e reduz a chance de inconsistências decorrentes de alterações não sincronizadas em várias partes da aplicação.

Redução de Duplicação

Ao aplicar o DRY, o Modelo atua como uma única fonte autoritativa para a lógica de negócios relacionada a dados. Isso elimina a duplicação de código e contribui para um código mais limpo e sustentável, o que também é benéfico.

Em resumo, a reutilização do código do Modelo, conforme promovida pelo princípio DRY, contribui para uma maior modularidade, consistência e facilidade de manutenção em uma aplicação. Essa abordagem é especialmente eficaz em situações em que há operações ou lógica de negócios comuns que podem ser aplicadas em diferentes partes da aplicação.

A preocupação de que "em implementações menos cuidadosas, os controladores podem se tornar monolíticos, assumindo muitas responsabilidades e perdendo a vantagem da separação de responsabilidades" destaca um desafio potencial na aplicação do padrão Model-View-Controller (MVC). Vamos explorar mais sobre esse aspecto?

Desafios dos controladores monolíticos

Assumir muitas responsabilidades

Os controladores, na arquitetura MVC, são responsáveis por gerenciar a lógica de controle, recebendo entradas da visualização, manipulando a lógica de negócios e atualizando o modelo. No entanto, há um desafio nisso, já que em implementações menos cuidadosas, pode haver uma tendência para que os controladores cresçam demais, assumindo múltiplas responsabilidades e se tornando monolíticos.

Perda da vantagem da separação de responsabilidades

A vantagem chave do MVC é a clara separação de responsabilidades entre Modelo, Visualização e Controlador. Quando os controladores se tornam monolíticos, essa separação é comprometida e a aplicação perde os benefícios da modularidade e da fácil manutenção.

Razões para controladores monolíticos

Falta de entendimento do MVC

Desenvolvedores e desenvolvedoras que não são familiares com os princípios do MVC podem não compreender completamente a necessidade de manter os controladores concisos. Isso pode levar à inclusão de muita lógica diretamente nos controladores.

Pressões de prazo

Em ambientes com prazos apertados ou pressões para entregar rapidamente, profissionais de desenvolvimento podem optar por implementações mais rápidas e menos modulares, contribuindo para controladores monolíticos.

Consequências dos controladores monolíticos

Dificuldade de manutenção

Controladores monolíticos são mais difíceis de manter e entender. A inclusão de muita lógica em um único controlador pode resultar em código confuso e propenso a erros.

Dificuldade em testes unitários

Controladores monolíticos podem ser desafiadores de testar devido à sua complexidade. Testes unitários podem se tornar menos eficazes, tornando a detecção de erros mais difícil.

Aumento do acoplamento

O aumento da lógica em um controlador pode levar a um maior acoplamento entre a lógica de controle e os detalhes da implementação, tornando mais difícil realizar alterações sem afetar outras partes do código.

Estratégias para evitar controladores monolíticos

Aplicar princípios SOLID

Aplicar princípios de design de software, como o SOLID, pode ajudar a evitar a criação de controladores monolíticos. Isso inclui a divisão de responsabilidades em classes e métodos específicos.

Utilizar serviços auxiliares

Separar lógica específica em serviços auxiliares pode ser uma estratégia eficaz. Esses serviços podem ser injetados no controlador, mantendo-o mais enxuto.

Revisões de código e boas práticas

Realizar revisões de código regulares e promover boas práticas de programação são fundamentais para evitar a criação de controladores monolíticos. A equipe deve estar ciente dos princípios do MVC e da importância da modularidade.

Em resumo, controladores monolíticos são um desafio potencial na implementação do padrão MVC. Evitar essa situação requer práticas de programação conscientes, compreensão profunda dos princípios do MVC e o compromisso de manter a modularidade e a separação de responsabilidades.

A observação "Em aplicações que requerem atualizações em tempo real intensivas, pode ser necessário adotar soluções adicionais para lidar eficientemente com a comunicação entre o Modelo e a View" destaca um desafio específico que pode surgir em situações nas quais a sincronização em tempo real entre o Modelo e a Visualização é crucial. Vejamos alguns pontos desse cenário.

Desafios em aplicações com atualizações em tempo real

Necessidade de atualizações imediatas

Em certos tipos de aplicações, como aplicações colaborativas em tempo real, painéis de monitoramento ou jogos online, a necessidade de atualizações em tempo real é crítica. Os usuários esperam ver alterações instantâneas refletidas na interface do usuário.

Comunicação assíncrona

O modelo padrão de comunicação entre o Modelo e a View em muitas implementações MVC é síncrono, ou seja, a View é atualizada quando o Modelo é modificado. Em cenários de atualização em tempo real, a comunicação assíncrona pode ser mais eficaz para garantir respostas rápidas.

Soluções adicionais para lidar com atualizações em tempo real

Sockets e comunicação em tempo real

A utilização de tecnologias de comunicação em tempo real, como WebSockets, pode ser uma solução eficaz. Isso permite uma comunicação bidirecional assíncrona entre o servidor (Modelo) e o cliente (View).

Eventos e observadores

Implementar um sistema de eventos ou observadores no Modelo, de modo que a View seja notificada automaticamente quando ocorrerem alterações relevantes. Isso permite uma atualização eficiente da interface do usuário sem a necessidade de verificações constantes.

Utilização de frameworks de atualização em tempo real

Em alguns casos, a utilização de frameworks ou bibliotecas específicas para atualização em tempo real pode simplificar a implementação. Exemplos incluem Socket.io para WebSockets ou Firebase Realtime Database.

Adoção de arquiteturas assíncronas

Em arquiteturas mais avançadas, como arquiteturas baseadas em mensagens ou sistemas de filas, a comunicação assíncrona pode ser facilitada. Isso permite a distribuição eficiente de eventos e atualizações.

Benefícios das soluções adicionais

Resposta imediata

A adoção de soluções adicionais para comunicação em tempo real permite que as atualizações sejam refletidas na interface do usuário quase que instantaneamente, melhorando a experiência do usuário.

Menor carga no servidor

Utilizar comunicação assíncrona pode reduzir a carga no servidor, pois as atualizações são enviadas apenas quando necessário, evitando consultas constantes por parte da View.

Maior escalabilidade

Soluções eficientes para comunicação em tempo real podem melhorar a escalabilidade da aplicação, permitindo que um grande número de clientes seja atendido simultaneamente.

Resumindo, nas situações em que atualizações em tempo real são críticas, é importante considerar soluções adicionais para a comunicação entre o Modelo e a View. A escolha da solução dependerá das necessidades específicas da aplicação e das tecnologias utilizadas.

O QUE VOCÊ VIU NESTA AULA?

Aprendemos sobre os segredos de uma API em ação utilizando a arquitetura MVC. Também tivemos conteúdo sobre DRY, além de nos aprofundarmos ainda mais em MVC.

EXEMPLO

REFERÊNCIAS

MARTIN, R. C. **A little about patterns.** 2014. Disponível em: <<https://blog.cleancoder.com/uncle-bob/2014/06/30/ALittleAboutPatterns.html>>. Acesso em: 13 de maio de 2024.

MVC design pattern. Geeks for geeks, 2024. Disponível em: <<https://www.geeksforgeeks.org/mvc-design-pattern/>>. Acesso em: 13 de maio de 2024.

MVC. mdn web docs, [s.d.]. Disponível em: <<https://developer.mozilla.org/en-US/docs/Glossary/MVC>>. Acesso em: 13 de maio de 2024.

PALAVRAS-CHAVE

Palavras-chave: MVC, DRY, Node.

EXEMPLO

POS TECH